
Cyart Technology

Security Assessment Findings Report

Web Application Penetration Testing and Vulnerability Assessment Report

Date - 23/5/2026

Version 1.0

By - Srishti Khare

Objectives.....	3
Scope.....	3
Environment Setup.....	4
Methodology.....	4
VULNERABILITY ASSESSMENT AND EXPLOITATION.....	7
SQL Injection.....	7
Insecure CAPTCHA.....	10
Reflected Cross-Site Scripting (XSS).....	12
VA by SQLMAP.....	15
Metasploit Advanced Exploitation Section.....	16
Metasploit Reconnaissance & Advanced Exploitation.....	16
Exploit-DB PoC Customization Section.....	18
Exploit-DB PoC Customization.....	18
Findings summary.....	20
Recommendations.....	21

Vulnerability Assessment and Penetration Testing (VAPT) is a security testing process used to identify, analyze and exploit vulnerabilities present within systems, applications and networks. The objective of VAPT is to evaluate the security posture of a target environment and recommend mitigations to reduce potential security risks.

This assessment was performed on OWASP Broken Web Applications (BWA) using Damn Vulnerable Web Application (DVWA) as the primary target platform. The testing focused on identifying common web application vulnerabilities aligned with the OWASP Top 10 security risks, including SQL Injection, Insecure CAPTCHA implementation and request manipulation vulnerabilities.

The assessment was conducted in a controlled virtual lab environment using Kali Linux, Burp Suite and SQLMap to perform both manual and automated testing techniques.

Objectives

The primary objectives of this assessment were:

- To identify vulnerabilities present within the target web application.
- To perform manual and automated penetration testing techniques.
- To analyze insecure authentication and input validation mechanisms.
- To demonstrate exploitation of web application vulnerabilities.
- To understand the impact of insecure coding practices.
- To recommend remediation techniques for identified vulnerabilities.

Scope

The scope of this assessment was limited to the OWASP Broken Web Applications (BWA) virtual machine hosted within a controlled virtualized environment. Testing activities were performed specifically against the Damn Vulnerable Web Application (DVWA) module.

The assessment included:

- Manual web application testing
- HTTP request interception and manipulation
- SQL Injection testing
- CAPTCHA bypass testing
- Automated SQL Injection exploitation using SQLMap

Denial-of-Service (DoS) attacks and destructive exploitation techniques were excluded from the scope of testing.

Environment Setup

The penetration testing lab environment was configured using Oracle VirtualBox virtualization software. The setup consisted of separate attacker and target virtual machines connected through a Host-Only network configuration.

Attacker Machine:

- Kali Linux
- Tools: Burp Suite, SQLMap, Firefox Browser

Target Machine:

- OWASP Broken Web Applications (BWA)
- DVWA (Damn Vulnerable Web Application)

Network Configuration:

- Host-Only Adapter
- Internal communication between attacker and target systems

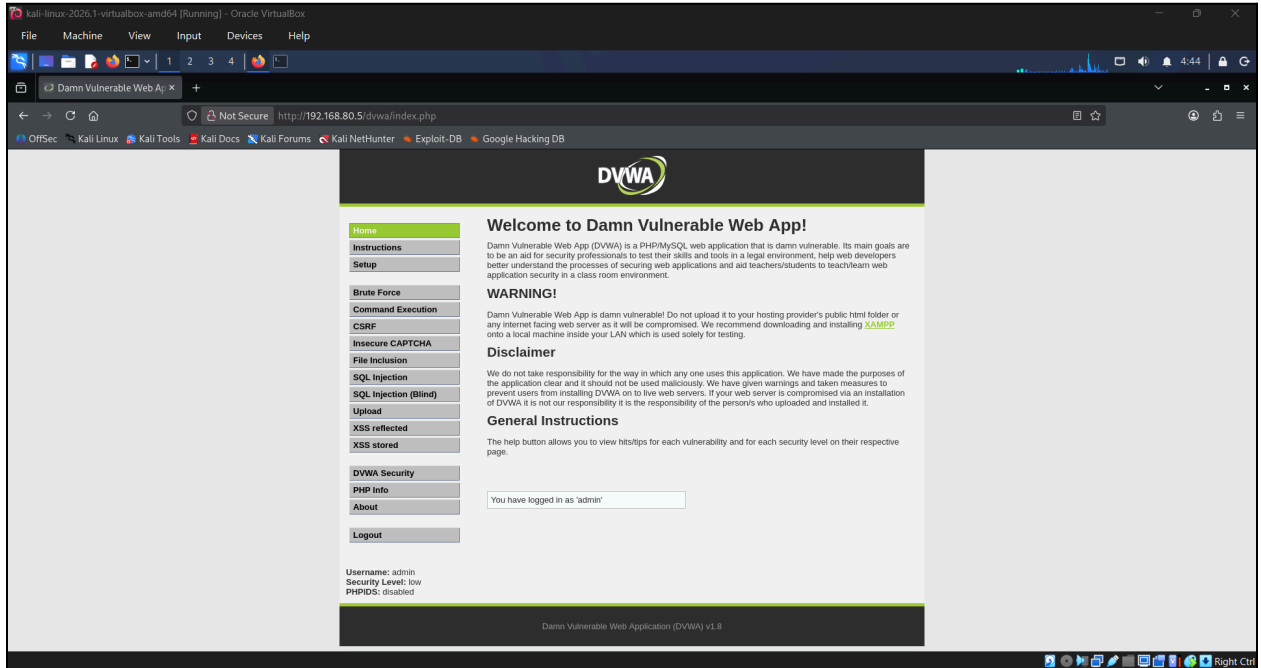
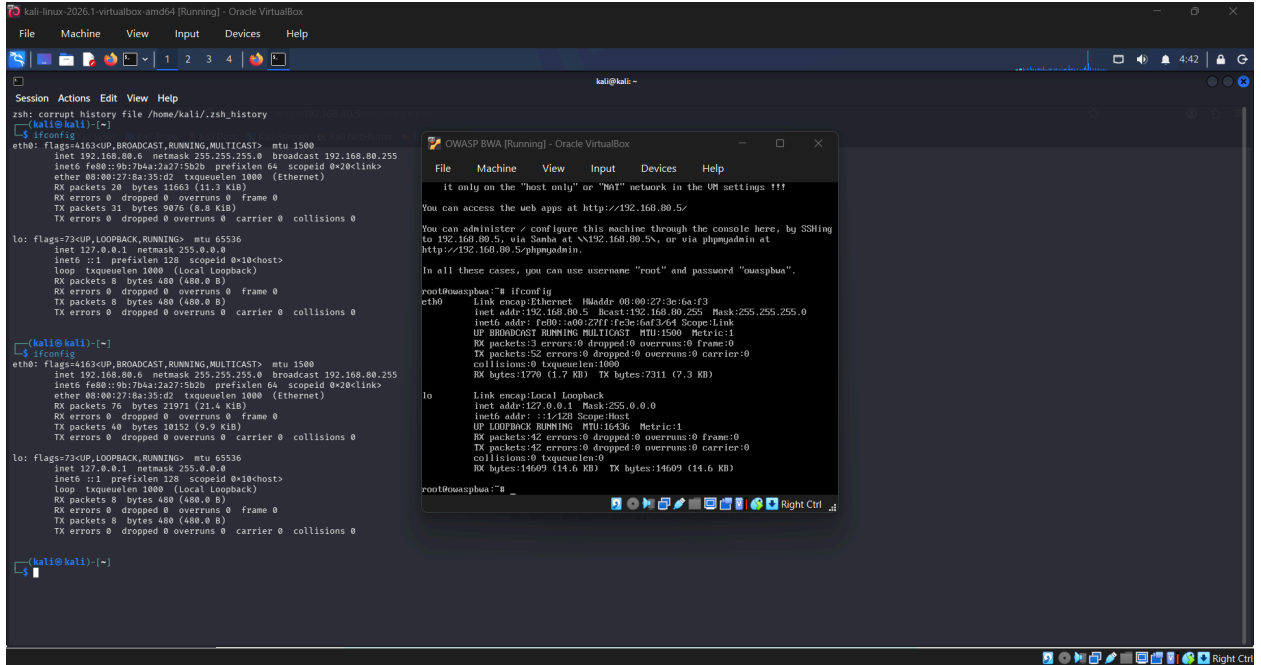
Methodology

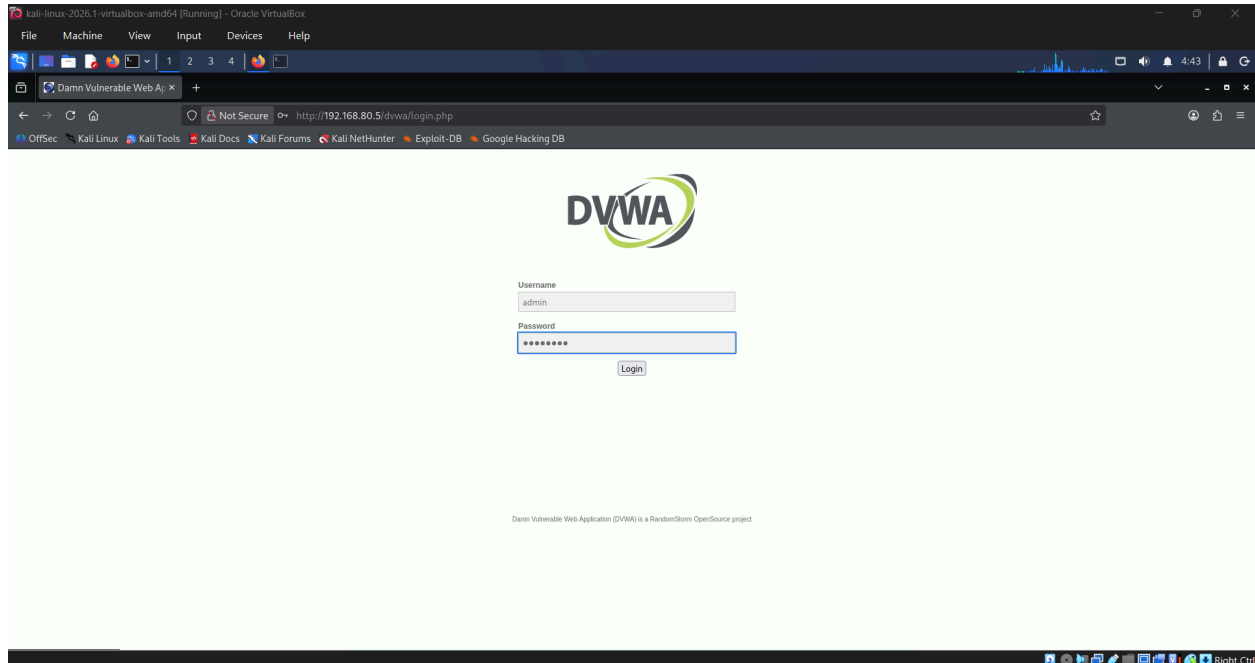
The assessment methodology followed a structured penetration testing approach aligned with OWASP Web Security Testing Guide (WSTG) and PTES principles.

The following phases were performed during the assessment:

1. Environment Configuration

The vulnerable target machine and attacker machine were configured within an isolated virtualized lab environment using VirtualBox.





2. Reconnaissance and Enumeration

The target system was identified and accessed through internal network communication. Basic connectivity and web application access were verified.

3. Vulnerability Assessment

Manual and automated testing techniques were performed to identify vulnerabilities within the DVWA application.

4. Exploitation

Identified vulnerabilities such as SQL Injection and Insecure CAPTCHA were exploited using Burp Suite and SQLMap.

5. Documentation and Reporting

All findings, exploitation steps, screenshots and remediation recommendations were documented within the assessment report.

VULNERABILITY ASSESSMENT AND EXPLOITATION

SQL Injection

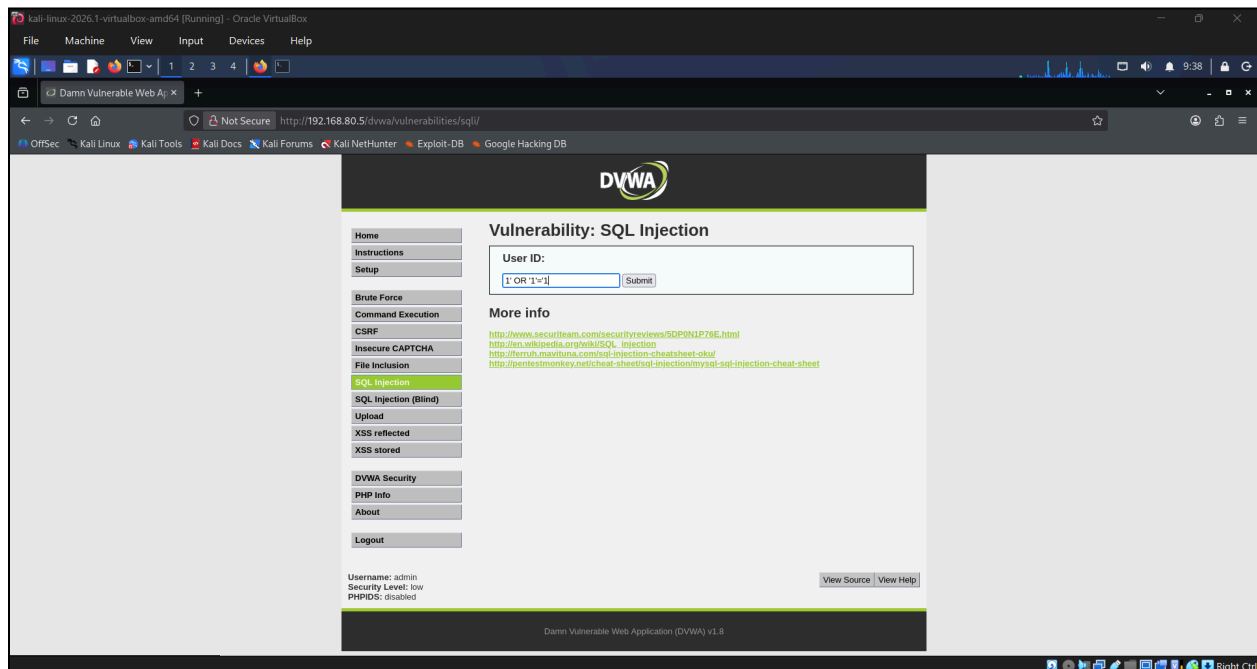
SQL Injection is a web application vulnerability that occurs when user-controlled input is improperly validated before being included within SQL queries. Attackers can manipulate backend database queries to extract sensitive information, bypass authentication or modify database contents.

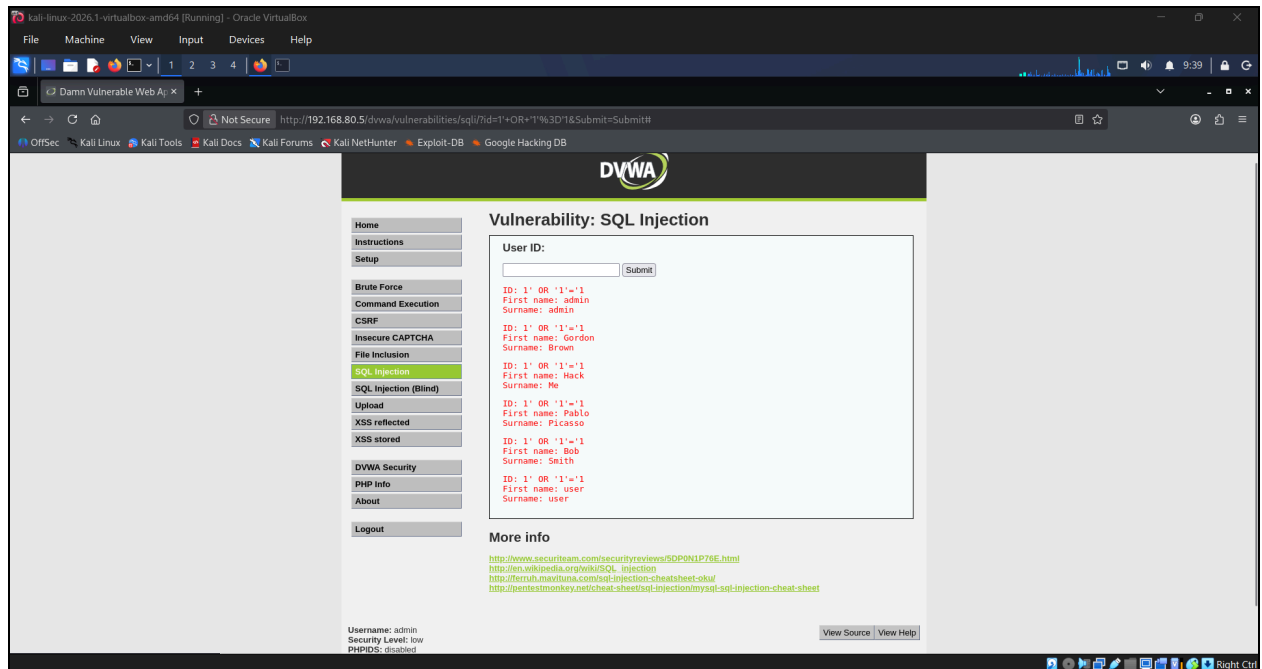
During the assessment, the SQL Injection vulnerability was identified within the DVWA SQL Injection module. Manual testing was performed by injecting malicious SQL payloads through the user input parameter.

Payload Used:

1' OR '1'='1

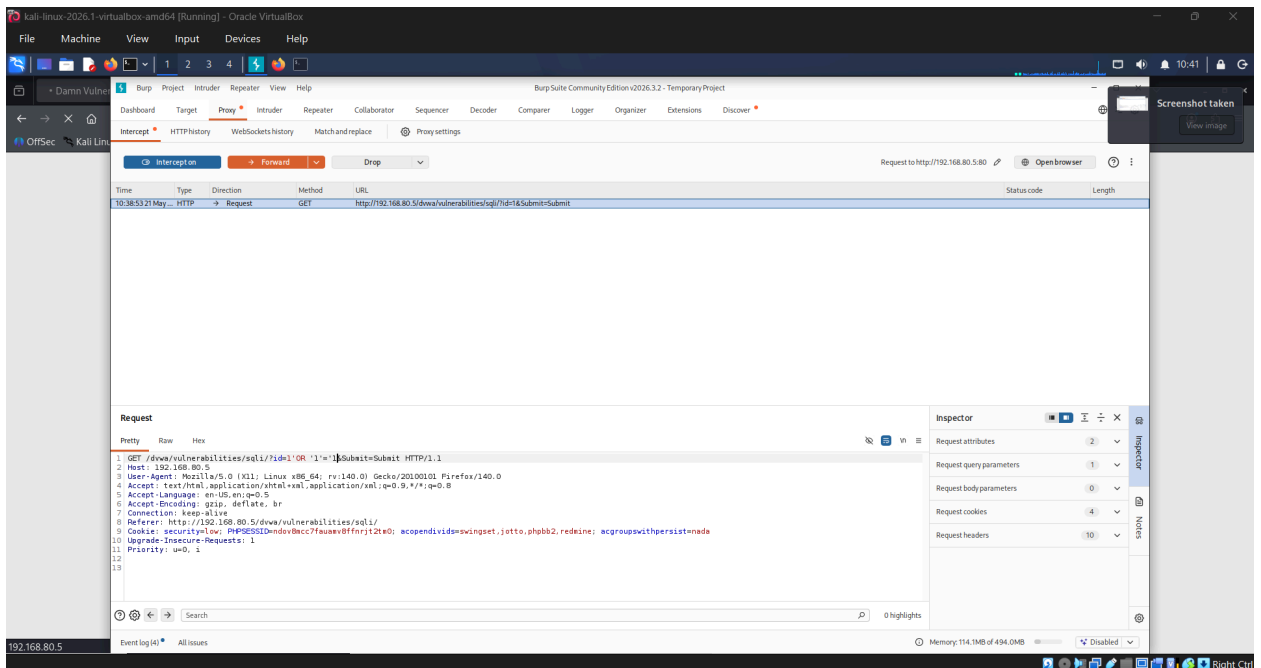
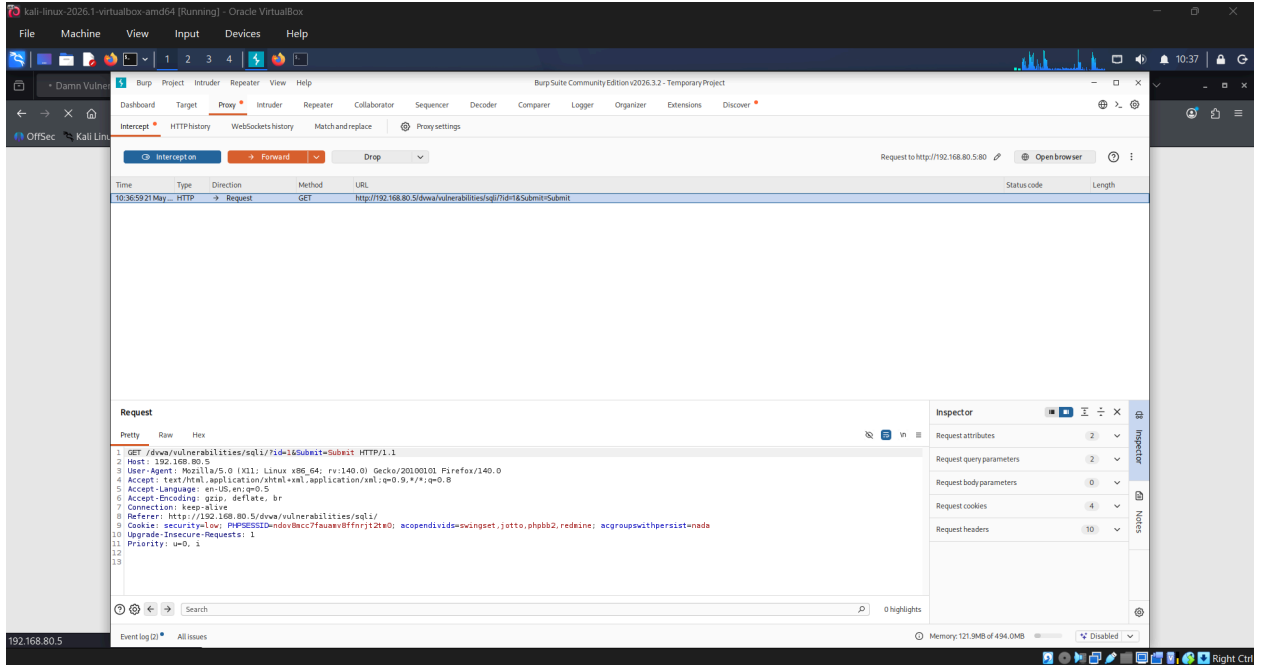
The payload successfully manipulated the backend SQL query and returned multiple database records, confirming the presence of SQL Injection vulnerability.

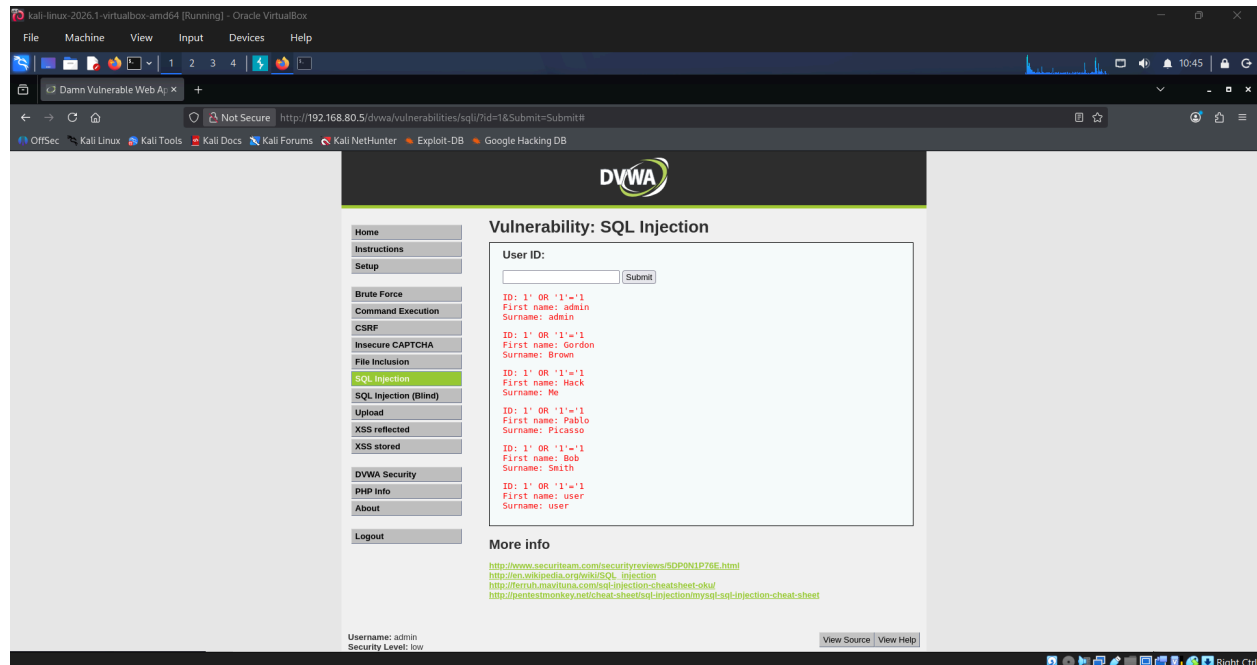




Field	Value
Vulnerability	SQL Injection
Severity	Critical
OWASP	A03:2021 Injection
Impact	Database disclosure & authentication bypass
Recommendation	Parameterized queries & input validation

BURP SUITE INTERCEPTION





Impact:

Successful exploitation of SQL Injection vulnerabilities may allow attackers to bypass authentication, extract sensitive database information and manipulate backend database queries.

Remediation:

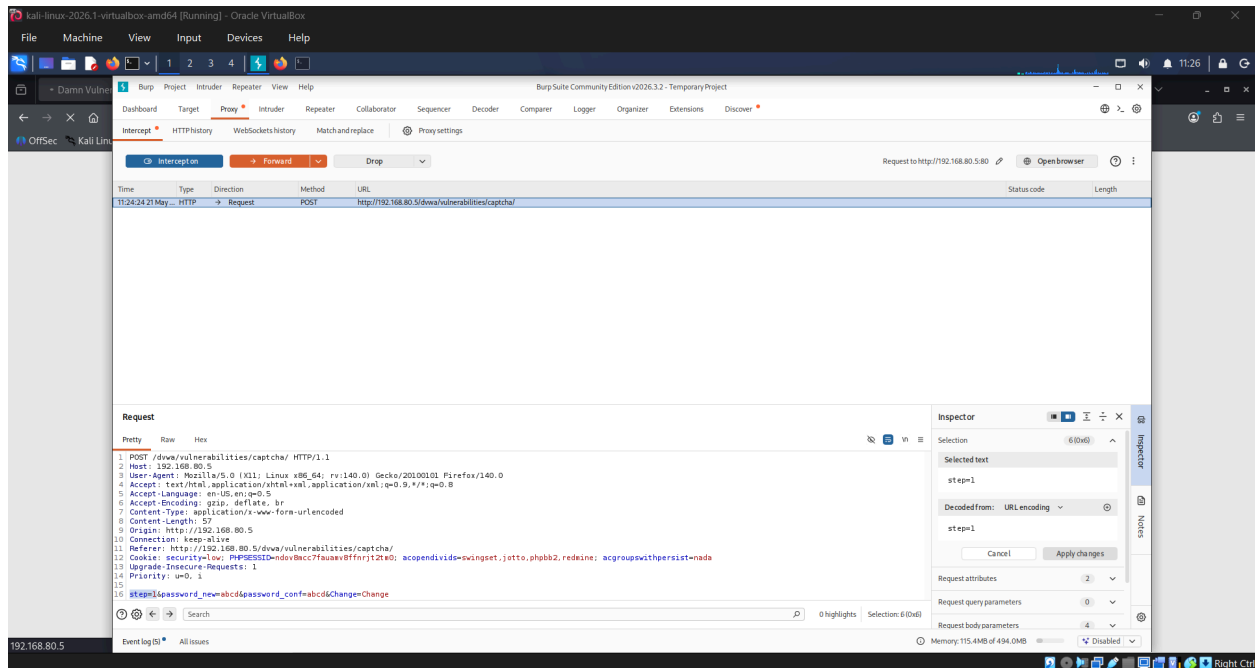
- Implement parameterized queries and prepared statements.
- Validate and sanitize all user inputs.
- Avoid dynamic SQL query construction.
- Implement Web Application Firewall (WAF) protections.

Insecure CAPTCHA

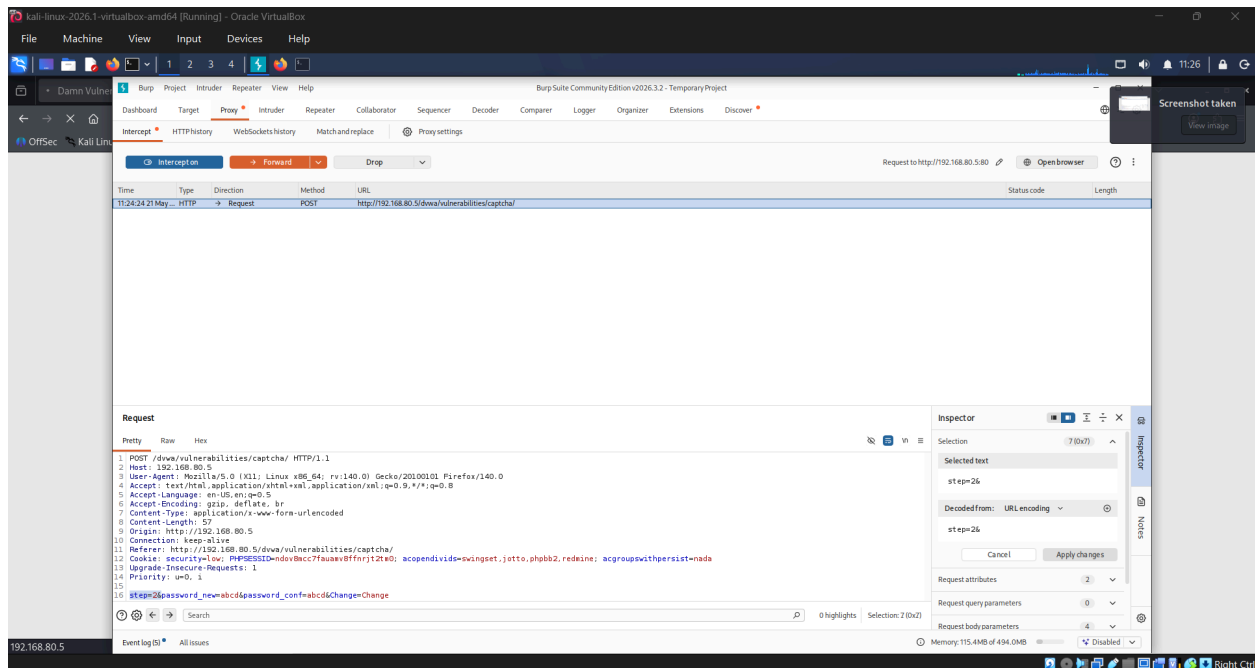
Insecure CAPTCHA vulnerabilities occur when CAPTCHA validation mechanisms are improperly implemented and rely on client-side validation instead of secure server-side verification.

During the assessment, the DVWA Insecure CAPTCHA module was tested using Burp Suite request interception. The application workflow parameter was manipulated from step=1 to step=2, bypassing the CAPTCHA verification process and allowing unauthorized password modification.

The vulnerability demonstrated improper trust in client-side parameters and insufficient server-side validation controls.



value changed to step=2



The CAPTCHA validation process was bypassed by modifying the application workflow parameter from step=1 to step=2, allowing unauthorized password change without proper CAPTCHA verification.

Impact:

Attackers may bypass CAPTCHA protections, automate malicious requests and perform unauthorized actions without proper verification.

Remediation:

- Implement strict server-side CAPTCHA validation.
- Avoid trusting client-side workflow parameters.
- Validate all application state transitions securely on the server.

Reflected Cross-Site Scripting (XSS)

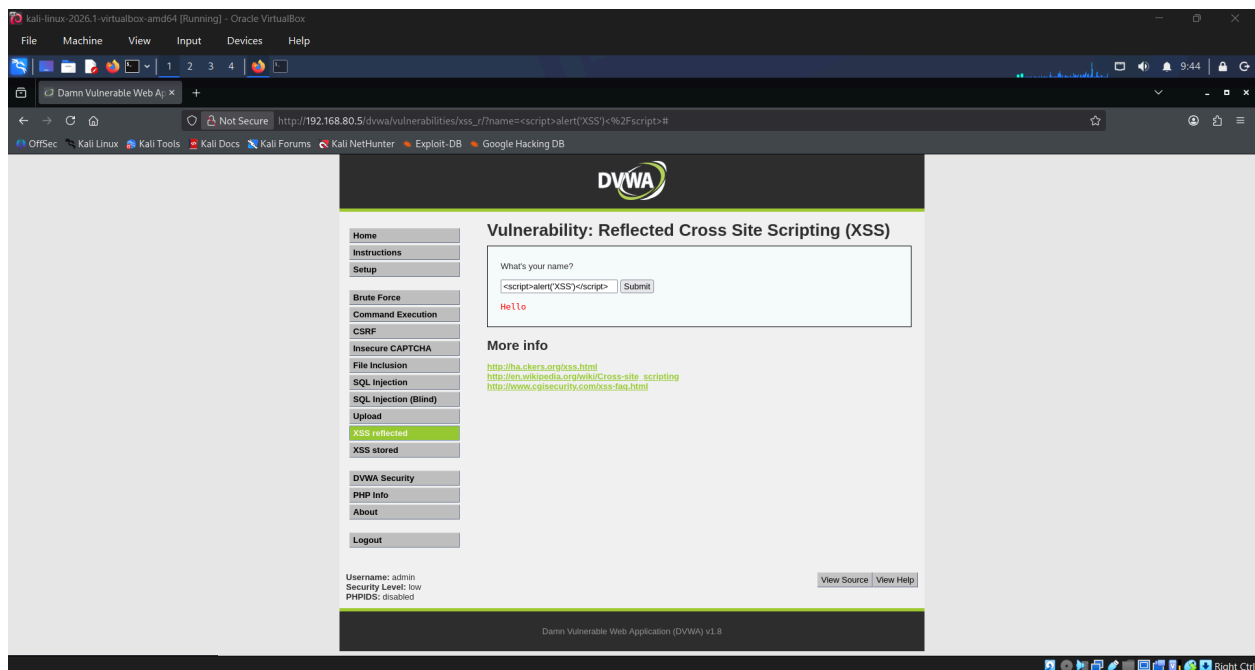
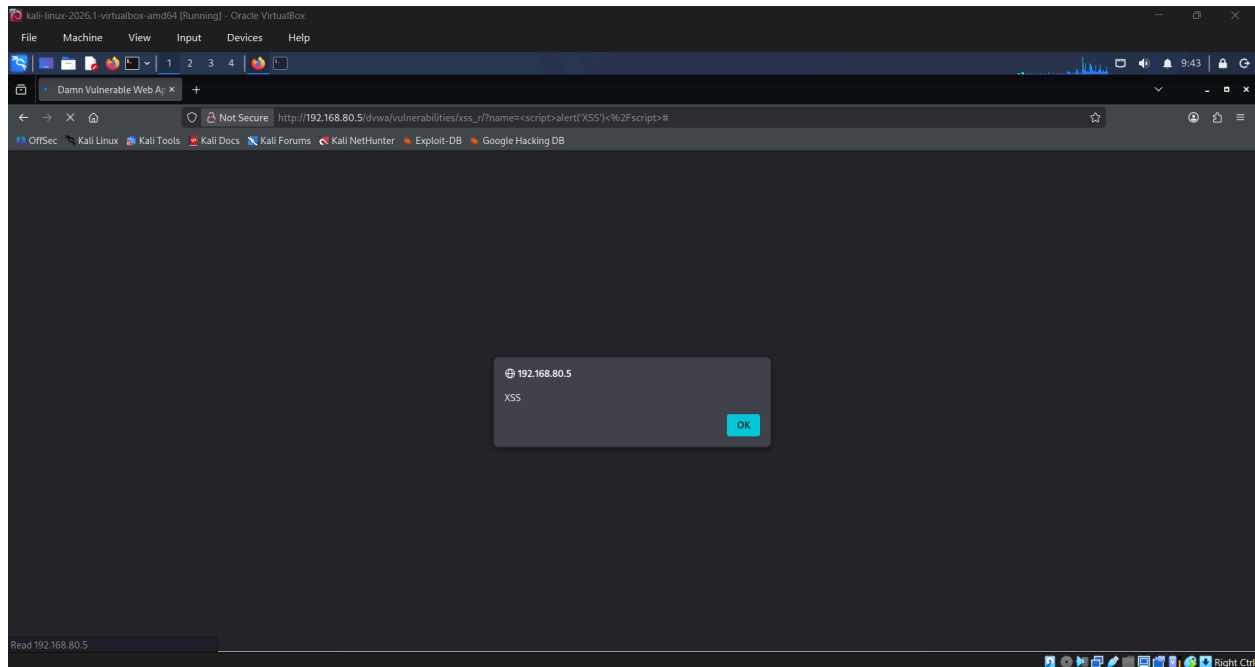
Reflected Cross-Site Scripting (XSS) is a web application vulnerability that occurs when user-supplied input is reflected back to the browser without proper sanitization or output encoding. Attackers can inject malicious JavaScript code which executes within the victim's browser session.

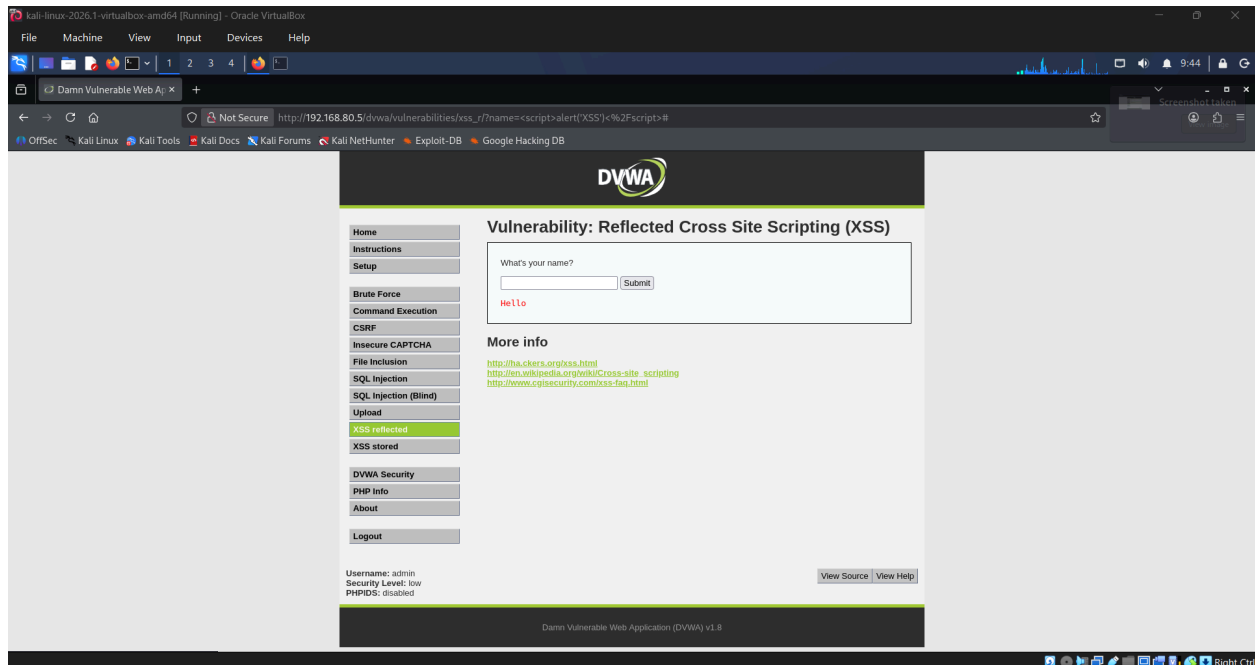
During the assessment, the DVWA Reflected XSS module was tested using manual payload injection techniques. A malicious JavaScript payload was inserted into the vulnerable input parameter to verify whether the application properly sanitized user input.

Payload Used:

```
<script>alert('XSS')</script>
```

Upon submission, the application executed the injected JavaScript code and generated a browser alert popup, confirming the presence of a Reflected XSS vulnerability.





Impact:

Successful exploitation of Reflected XSS vulnerabilities may allow attackers to execute malicious scripts within a victim's browser session, steal session cookies, hijack user sessions and redirect users to malicious websites.

Remediation:

- Implement proper input sanitization and output encoding.
- Use Content Security Policy (CSP) headers.
- Validate and filter user-controlled input.
- Avoid reflecting unsanitized user input within application responses.

Field	Value
Vulnerability	Reflected XSS
Severity	Medium
OWASP Category	A03:2021 Injection
Impact	Client-side script execution
Recommendation	Input sanitization & output encoding

VA by SQLMAP

```
kali-linux-2026.1-virtualbox-amd64 [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

[+] [kali@kali:~]$ sqlmap -u "http://192.168.80.5/dvwa/vulnerabilities/sql/7?id=1&Submit=Submit" --cookie="security=low; PHPSESSID=ndovBmcc7faaam8ffncrj2tmo" --batch --dbs
[+] [kali@kali:~]$ https://sqlmap.org

[+] starting @ 14:51:31 / 2026-05-21/

[14:51:31] [INFO] testing connection to the target URL
[14:51:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[14:51:32] [INFO] testing if the target URL content is stable
[14:51:32] [INFO] target URL content is stable
[14:51:32] [INFO] testing if GET parameter 'id' is dynamic
[14:51:32] [WARNING] GET parameter 'id' does not appear to be dynamic
[14:51:32] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[14:51:32] [INFO] heuristic (XSS) test shows that GET parameter 'id' might be vulnerable to cross-site scripting (XSS) attacks
[14:51:32] [INFO] testing for SQL injection on GET parameter 'id'
[14:51:32] [INFO] it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
[14:51:32] [INFO] for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (2) and risk (1) values? [Y/n] Y
[14:51:32] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[14:51:32] [WARNING] reflective values(s) found and filtering out
[14:51:32] [INFO] testing 'boolean-based blind - Parameter replace (original value)'
[14:51:32] [INFO] testing 'Generic inline queries'
[14:51:32] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[14:51:32] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (MySQL comment)'
[14:51:32] [INFO] testing 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)'
[14:51:32] [INFO] GET parameter 'id' appears to be 'OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)' injectable (with --not-string='Me')
[14:51:32] [INFO] testing 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[14:51:32] [INFO] GET parameter 'id' is 'MySQL > 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)' injectable
[14:51:32] [INFO] testing 'MySQL inline queries'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 stacked queries (comment)'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 stacked queries'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP - comment)'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 stacked queries (query SLEEP)'
[14:51:32] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[14:51:32] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)'
[14:51:32] [INFO] testing 'MySQL > 5.0.12 AND time-based blind (query SLEEP)' injectable
[14:51:32] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[14:51:32] [INFO] testing 'MySQL UNION query (NULL) - 1 to 20 columns'
[14:51:32] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[14:51:32] [INFO] target URL appears to have 2 columns in query
[14:51:32] [INFO] GET parameter 'id' is 'MySQL UNION query (NULL) - 1 to 20 columns' injectable
[14:51:32] [WARNING] in OR boolean-based injection cases, please consider usage of switch --drop-set-cookie' if you experience any problems during data retrieval

[+] ending @ 14:51:47 / 2026-05-21/

[+] [kali@kali:~]$

[+] [kali@kali:~]$
```

SQL Injection using SQLMap

- Automated SQL Injection testing was performed using SQLMap against DVWA.
- The id parameter was identified as vulnerable.
- SQLMap successfully detected:
 - Boolean-Based Blind SQL Injection
 - Error-Based SQL Injection

- Time-Based Blind SQL Injection
- UNION-Based SQL Injection
- Backend database identified as MySQL.
- Databases such as dvwa and information_schema were successfully enumerated.

Tool Used:

- SQLMap

Severity:

- Critical

Impact:

- Attackers may extract database contents, bypass authentication, and execute unauthorized database queries.

Recommendation:

- Use parameterized queries / prepared statements.
- Implement input validation and WAF protections.
- Avoid dynamic SQL query construction.

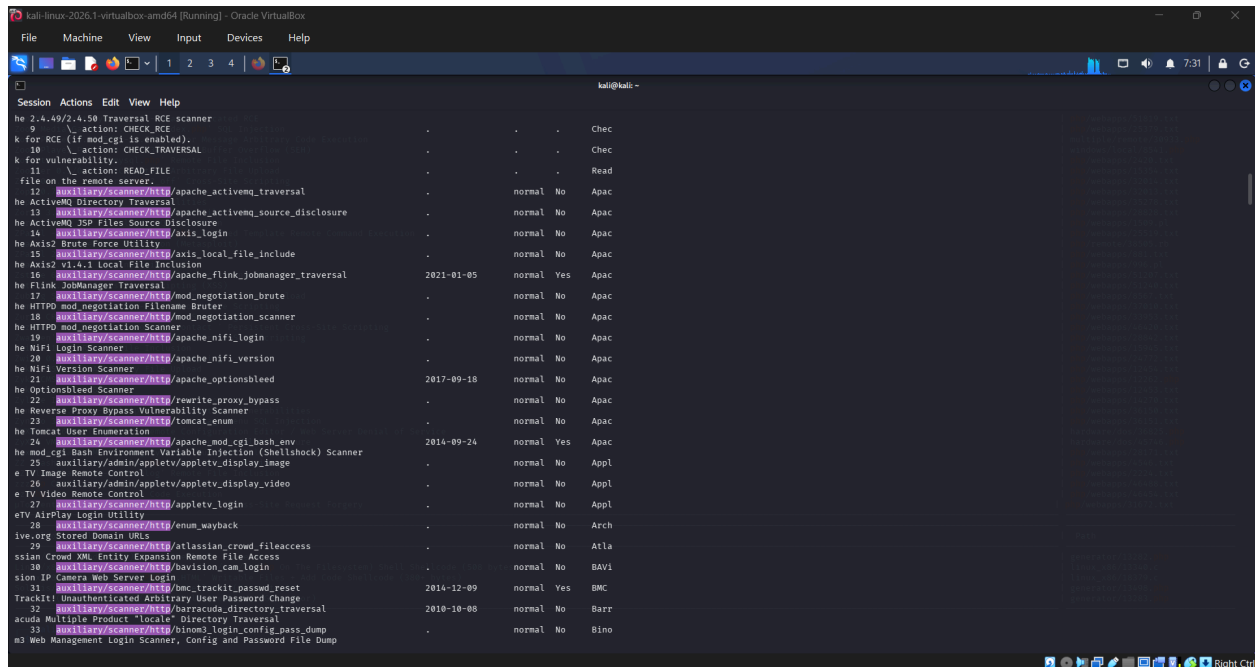
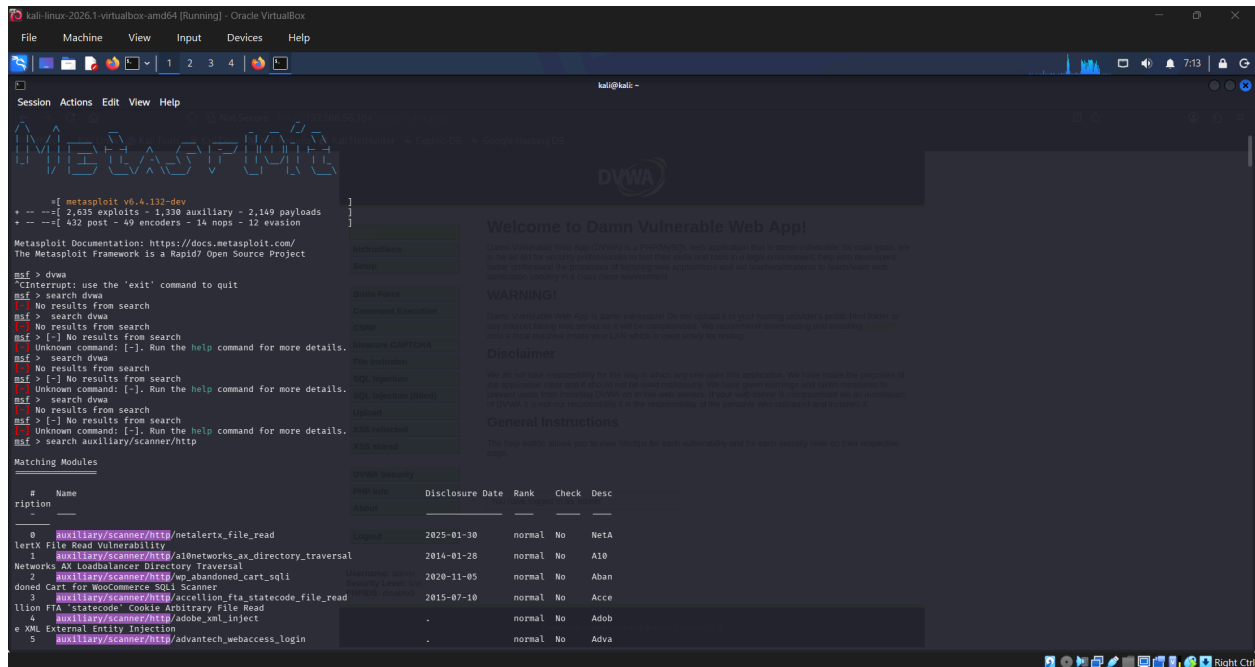
Metasploit Advanced Exploitation Section

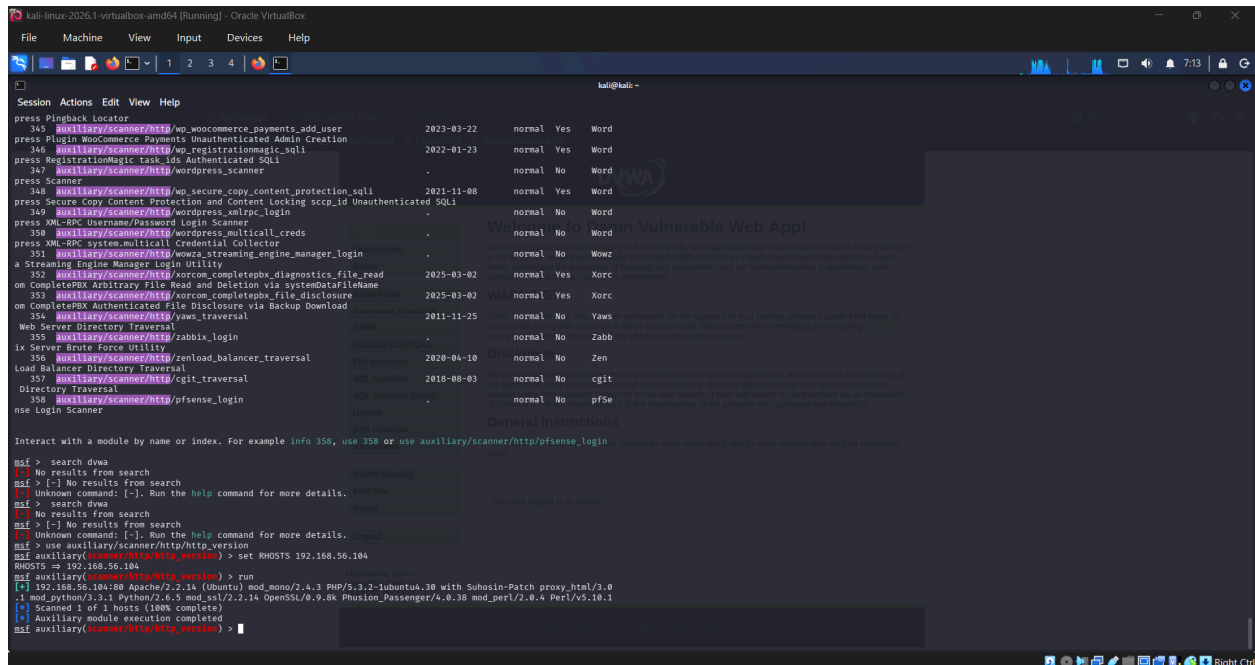
Metasploit Reconnaissance & Advanced Exploitation

Metasploit Framework was used to perform advanced reconnaissance and service enumeration against the OWASP BWA target environment. Auxiliary scanner modules were utilized to identify HTTP services, Apache version details, enabled server modules and accessible application directories.

The HTTP version scanner successfully identified Apache 2.2.14 running on Ubuntu with PHP 5.3.2 and additional enabled modules such as OpenSSL, Perl and Python integrations. Such information disclosure can assist attackers in identifying vulnerable software versions and planning exploitation attempts.

Additionally, directory enumeration was performed using the Metasploit HTTP directory scanner module to identify accessible web application paths hosted on the target system. The activity demonstrated practical usage of Metasploit for reconnaissance, enumeration and post-exploitation intelligence gathering.



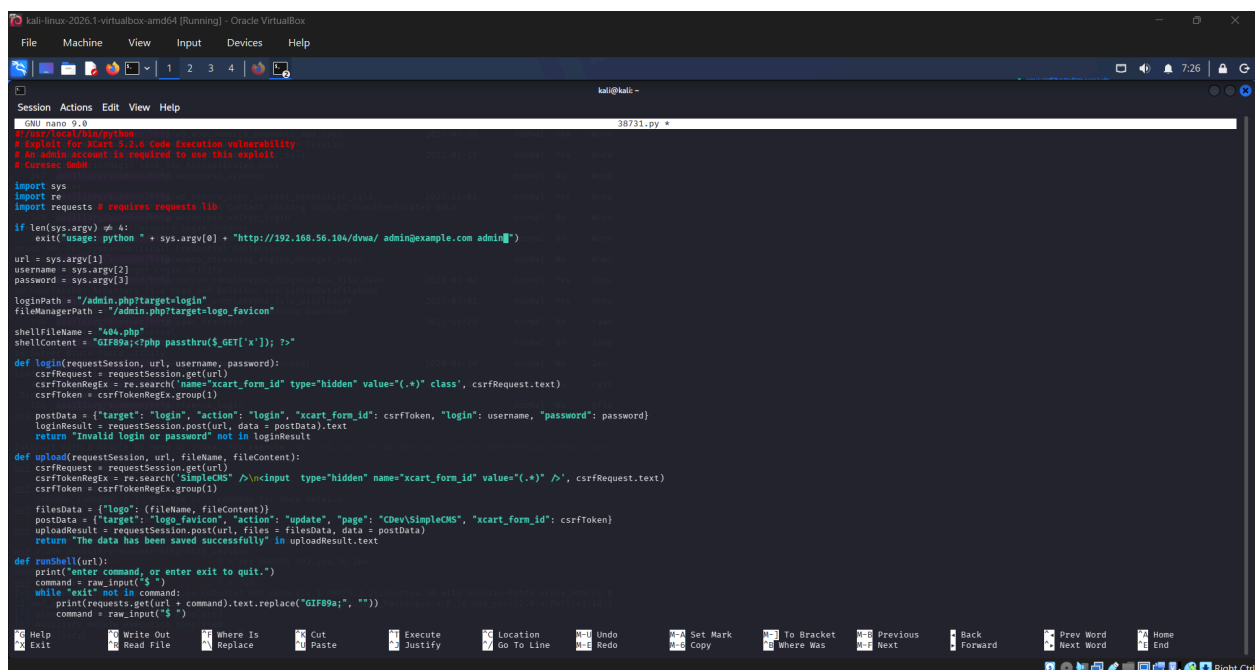
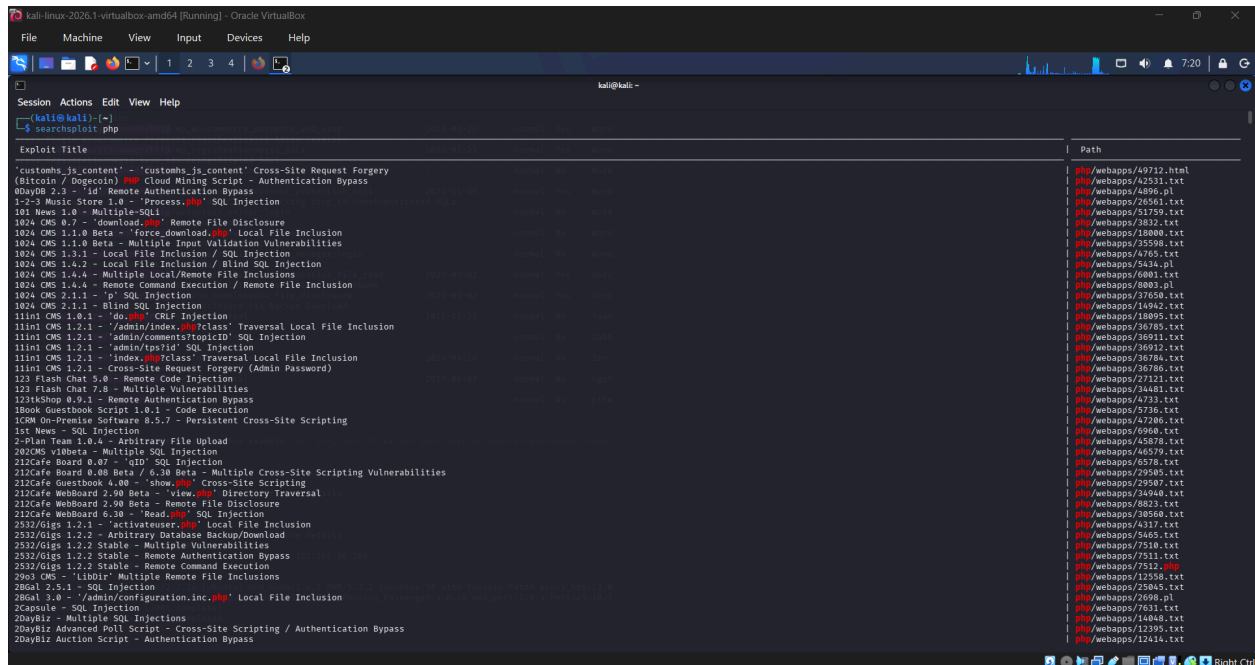


Exploit-DB PoC Customization Section

Exploit-DB PoC Customization

A publicly available Python Proof-of-Concept (PoC) exploit obtained from Exploit-DB was reviewed and customized for the OWASP BWA testing environment. The exploit script parameters and target configuration values were modified to align with the local virtual lab environment.

The customization activity demonstrated understanding of exploit adaptation and environment-specific exploit modification techniques commonly used during penetration testing engagements. SearchSploit utility available in Kali Linux was used to identify and retrieve the exploit script from the Exploit-DB repository.



The original Python Proof-of-Concept exploit script was modified by updating the target IP address and request parameters to align with the OWASP BWA testing environment. These modifications demonstrated basic exploit customization techniques and highlighted how publicly available exploits can be adapted for environment-specific penetration testing activities.

Findings summary

The Vulnerability Assessment and Penetration Testing (VAPT) exercise conducted on the OWASP BWA and DVWA environment successfully identified multiple web application security weaknesses aligned with OWASP Top 10 categories. The assessment included reconnaissance, service enumeration, traffic analysis, manual exploitation, automated exploitation and exploit customization activities.

Critical vulnerabilities such as SQL Injection were successfully exploited to demonstrate unauthorized database access and data extraction. Cross Site Scripting (XSS) vulnerabilities were identified and validated through payload execution in the browser environment. Insecure CAPTCHA implementation weaknesses were also exploited through parameter manipulation and authentication workflow bypass techniques.

Automated exploitation using sqlmap confirmed the presence of multiple SQL injection techniques including boolean-based, error-based, time-based and UNION query injection. Metasploit Framework was utilized for advanced reconnaissance and service enumeration to identify server configurations and accessible application paths.

The assessment highlighted improper input validation, insecure server-side validation mechanisms, excessive information disclosure and insecure application configurations. These weaknesses can potentially allow attackers to compromise application confidentiality, integrity and availability if exploited in real-world environments.

Finding ID	Vulnerability	Severity	Affected Component	Risk Description	Status
F-01	SQL Injection	Critical	DVWA SQLi Module	Attackers can manipulate SQL queries to access, modify or extract sensitive database information.	Exploited
F-02	Cross Site Scripting (XSS Reflected)	High	DVWA XSS Module	Malicious scripts can execute within the victim browser leading to session hijacking or phishing attacks.	Exploited

F-03	Insecure CAPTCHA Validation	Medium	DVWA CAPTCHA Module	CAPTCHA workflow bypass through parameter manipulation may allow unauthorized password reset attempts.	Exploited
F-04	Information Disclosure	Medium	Apache HTTP Service	Server banners and version disclosure reveal technologies and software versions useful for attackers.	Identified
F-05	Directory Enumeration Exposure	Medium	Web Server Directories	Accessible directories and application paths may assist attackers during reconnaissance activities.	Identified
F-06	Weak Input Validation	High	Multiple Web Inputs	Lack of proper server-side validation increases the likelihood of injection and client-side attacks.	Identified
F-07	Automated SQL Exploitation	Critical	Database Layer	sqlmap successfully automated SQL injection exploitation and database enumeration activities.	Exploited
F-08	Exploit Adaptation Exposure	Low	Public PoC Usage	Publicly available exploit scripts can be customized and adapted against vulnerable environments.	Demonstrated

Recommendations

The following remediation measures are recommended to improve the security posture of the tested web application environment:

- Implement parameterized queries and prepared statements to prevent SQL Injection vulnerabilities.
- Validate and sanitize all user-controlled input before processing requests.
- Implement proper output encoding techniques to mitigate Cross-Site Scripting (XSS) vulnerabilities.
- Enforce strict server-side validation mechanisms for CAPTCHA and authentication workflows.
- Avoid trusting client-side parameters and application state values.
- Implement secure session handling and cookie protection mechanisms.
- Deploy Web Application Firewall (WAF) protections to identify and block malicious requests.
- Perform regular security assessments and vulnerability testing activities.

```

kali@kali: ~
Session Actions Edit View Help

Zpanel - Remote Code Execution (Metasploit)
Zpanel 2.5 - SQL Injection
Zpanel 2.5b10 - SQL Injection
Zstore 6.5.4 - Reflected Cross-Site Scripting (XSS)
Zstore 6.6.0 - Cross-Site Scripting (XSS)
Zubrag Smart File Download 1.3 - Arbitrary File Download
Zumset.com FbiLike 1.00 - 'id' Cross-Site Scripting
Zurmo CRM - Persistent Cross-Site Scripting
Zuz Music 2.1 - 'zuzconsole/...contact' Persistent Cross-Site Scripting
Zwahlen's Online Shop 5.2.2 - 'cat' Cross-Site Scripting
Zwiki 2.1.1 - Remote File Inclusion
Zwiki 0.10/0.36.2 - Cross-Site Scripting
Zyke CMS 1.0 - Arbitrary File Upload
Zyke CMS 1.1 - Authentication Bypass
Zyke CMS 1.1 - Bypass
Zyloze IT - Multiple Blind SQL Injections
Zyncro 3.0.1.20 - Multiple HTML Injection Vulnerabilities
Zyncro 3.0.1.20 - Social Network Message Menu SQL Injection
ZYXEL P-660HN-T11 IPv6 - Remote Configuration Editor / Web Server Denial of Service
ZYXEL VM3312-B100 - 1.00(AAP.?) - Credential Disclosure
Zyware Health Monitoring System - Multiple Vulnerabilities
ZZ FlashChat 3.1 - 'help.php' Local File Inclusion
ZZFlashChat 3.1 - 'adminlog' Remote File Inclusion
zzz.php CMS 1.6.1 - Cross-Site Request Forgery
zzz.php CMS 1.6.1 - Remote Code Execution
uTorrent (uTorrent) WebUI 0.310 Beta 2 - Cross-Site Request Forgery

Shellcode Title Path
Linux/x86 - Bind (/TCP) Shell Shellcode (Generator) generator/12822.php
Linux/x86 - Reverse (php) (Writes To /var/www/cb.php On The Filesystem) Shell Shellcode (500 bytes) linux_x86/13340.c
Linux/x86 - Search For '...php'/'...HTML' Writable Files + Add Code Shellcode (380+ bytes) linux_x86/18379.c
Solaris/x86 - Bind (/TCP) Shell Shellcode (Generator) generator/13490.php
Windows (XP SP1) - Bind (/TCP) Shell Shellcode (Generator) generator/12883.php

(kali@kali)~$ searchsploit -m php/remote/38731.py
Exploit: XCart 5.2.6 - Code Execution
URL: https://www.exploit-db.com/exploits/38731
Path: /usr/share/exploitdb/exploits/php/remote/38731.py
Codes: 0X00B-138416
Verified: False
File type: Python script, ASCII text executable
Copied to: /home/kali/38731.py

(kali@kali)~$

```

